CiT Backend Engineering Track

Week 2: Engineering Foundations & Excellence

Building the Habits, Mindset & Skills That Make Great Engineers

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 2
Instructor	Victor Kakama
Duration	6 months | 24 weeks
Tech Stack	Java 21, Spring Boot, PostgreSQL, Docker

Confidential — For CiT Trainees Only

Opening Prayer

As a Christ-centred community, we begin every session by inviting God's presence, wisdom, and guidance into our learning. Please stand or bow your heads as we pray together.

Prayer of Dedication

Heavenly Father,

We come before You at the start of this session with grateful hearts. You have given us minds capable of learning, hands capable of building, and a community to grow with. We do not take that lightly.

This week, we learn the foundations of excellent engineering — how to think clearly, write clean code, and work with integrity. We ask that You give us humility to learn, patience to practise, and wisdom to know that skill without character is incomplete.

As Proverbs 4:7 teaches us: "The beginning of wisdom is this: Get wisdom. Though it cost all you have, get understanding." Lord, let us be students who truly pursue understanding, not just answers.

Guide our instructor, bless our fellow trainees, and may everything we build this season serve others and bring honour to Your name.

In Jesus' name we pray,
Amen.

Scripture for the Week

"By wisdom a house is built, and through understanding it is established; through knowledge its rooms are filled with rare and beautiful treasures."

— Proverbs 24:3–4 (NIV)

Table of Contents

1. Welcome to Week 2
2. How to Think Like an Engineer
3. Code Quality Principles
4. Reading Documentation & Searching Effectively
5. Time Management & Professional Communication
6. Git Fundamentals
7. Key Takeaways & Next Steps

1. Welcome to Week 2

Welcome back! Last week you set up your environment and got your first taste of how the internet works. This week is different — we are not going to write much code. Instead, we are going to build something more important: the engineering mindset.

The engineers who stand out in any team are not necessarily the ones who type the fastest. They are the ones who think clearly, communicate proactively, write code their teammates can read, and know how to work through problems without panicking.

Everything you learn this week will make you a better engineer for the remaining 22 weeks — and for your entire career.

What to Expect This Week

By the end of this session, you will be able to:

- Break any complex problem into small, solvable steps
- Write pseudocode to plan your logic before touching a keyboard
- Apply DRY, meaningful naming, and clean code habits immediately
- Find answers in documentation without always asking someone else
- Communicate your progress professionally when you are blocked
- Use Git to track your code: init, add, commit, push, pull, branch, and open a pull request

2. How to Think Like an Engineer

Engineering is fundamentally about solving problems. The difference between a beginner and an experienced engineer is not just knowledge of syntax — it is the ability to approach problems systematically.

2.1 Problem Decomposition

The single most important skill in engineering is the ability to break a large, scary problem into small, manageable pieces. This is called problem decomposition.

No matter how complex a system is, it is made up of small, understandable parts. Your job as an engineer is to identify those parts and solve them one at a time.

Beginner's Example: Problem Decomposition

Problem: Build a user login feature.

A beginner looks at this and says: "I don't know where to start."

An engineer breaks it down:

- Show a login form with email and password fields
- Validate that both fields are filled in
- Send the email and password to the server
- Check if the email exists in the database
- Check if the password matches
- If yes: create a session/token and redirect to the dashboard
- If no: show an error message

Each bullet is now a solvable step. Tackle them one at a time. You will never be stuck on "login" again — you will only be stuck on one small step, and that is much easier to fix.

2.2 Algorithmic Thinking & Pseudocode

An algorithm is just a sequence of steps that solves a problem. Before writing real code, great engineers write pseudocode — a plain-English description of what the code will do. This lets you focus on the logic without worrying about syntax.

Pseudocode Rules

Pseudocode has no strict rules — it just needs to be clear. Use plain English. Indent to show structure. Focus on logic, not syntax.

Beginner's Example: Pseudocode for a Mobile Money Transfer

Task: Write the logic for transferring money between two accounts.

Pseudocode:

```
FUNCTION transferMoney(senderPhone, receiverPhone, amount):
  CHECK that amount is greater than 0
  FIND sender account by senderPhone
  IF sender account not found:
    RETURN error "Sender not found"
  IF sender balance < amount:
    RETURN error "Insufficient funds"
  FIND receiver account by receiverPhone
  IF receiver account not found:
    RETURN error "Receiver not found"
  SUBTRACT amount from sender balance
  ADD amount to receiver balance
  SAVE both accounts
  SEND confirmation SMS to both parties
  RETURN success "Transfer complete"
```

Notice: We have not written a single line of Java yet. But the entire problem is solved. Writing the actual code is now just translation.

2.3 The Debugging Mindset

Every engineer spends a large portion of their time debugging — finding and fixing problems. The secret is not to be lucky; it is to be systematic.

Beginner Approach (Avoid)	Engineering Approach (Adopt)
Change random things and hope it works	Read the error message carefully — it usually tells you exactly what is wrong
Panic when something breaks	Expect things to break. Breaking is how you learn where the edges are
Ask for help immediately without trying	Try for at least one hour using documentation before asking (the 1-Hour Rule)
Delete code when stuck	Comment out code to isolate the problem section
Give up and copy code you do not understand	Understand every line before you move forward

Exercise 1: Problem Decomposition Practice

Break each of the following features into at least 5 small steps using pseudocode or a numbered list. Work individually first, then share with a partner.

Feature 1: A user can reset their password by entering their email address.

Feature 2: An admin can view a report of all transactions made in the last 7 days.

Feature 3: A rider requests a boda boda, and the nearest available driver is notified.

Remember: There are no wrong answers here. The goal is to practise thinking in steps before touching code.

3. Code Quality Principles

Code is read far more often than it is written. In a professional team, your code will be reviewed by colleagues, maintained by your future self, and built upon by others. Writing readable, clean code is not optional — it is a core professional skill.

3.1 DRY — Don't Repeat Yourself

The DRY principle states: every piece of knowledge should have a single, unambiguous representation in the system. In practice, this means: if you find yourself writing the same code twice, that code belongs in a function.

Beginner's Example: DRY in Java

BAD — Repeated code:

```
// Calculating discounted price for three products
double price1 = 5000 - (5000 * 0.10);
double price2 = 8000 - (8000 * 0.10);
double price3 = 12000 - (12000 * 0.10);
```

GOOD — DRY code:

```
double applyDiscount(double price, double rate) {
    return price - (price * rate);
}

double price1 = applyDiscount(5000, 0.10);
double price2 = applyDiscount(8000, 0.10);
double price3 = applyDiscount(12000, 0.10);
```

Now if the discount logic changes, you update it in ONE place — not three.

3.2 Meaningful Naming

Good names are the cheapest form of documentation. A well-named variable or function tells a reader exactly what it does without them having to read the body.

Category	Bad Name	Good Name
Variable	d	transactionDate
Variable	x	accountBalance
Variable	flag	isTransactionApproved
Function	doStuff()	calculateMonthlyInterest()
Function	handleData()	validateUserRegistration()
Class	Manager	LoanApplicationService

3.3 Clean Code Habits

Clean code is not about clever tricks — it is about making your intent obvious to every reader.

Clean Code Checklist

- Functions do ONE thing — if a function does many things, split it up
- Functions are short — if it scrolls off the screen, it is too long
- No magic numbers — replace 0.15 with a named constant: `final double VAT_RATE = 0.15;`
- Comment the "why", not the "what" — the code already shows what it does
- Consistent formatting — use your IDE's auto-formatter, every time
- Delete dead code — do not comment out old code and leave it; delete it (Git has your history)

3.4 Introduction to SOLID Principles

SOLID is a set of five design principles that make code easier to understand, maintain, and extend. You will use these throughout your career. This week we introduce them — you will practise them deeply as we build real projects.

Principle	What It Means in Plain English
S — Single Responsibility	A class or function should do ONE thing. If you have to use the word "and" to describe what it does, split it up.
O — Open/Closed	Code should be open to extension but closed to modification. Add new features by adding new code, not by editing existing working code.
L — Liskov Substitution	A child class should be usable wherever a parent class is expected, without breaking anything.
I — Interface Segregation	Do not force a class to implement methods it does not need. Keep interfaces small and focused.
D — Dependency Inversion	Depend on abstractions (interfaces), not concrete implementations. This makes your code easier to test and change.

Exercise 2: Code Quality Refactoring

Look at the following code and identify all the code quality issues. Then rewrite it following the principles from this section.

Before (find the problems):

```
public class A {
    public void x(String e, String p, String n, String ph) {
        if (e != null && e.contains("@") && p.length() > 5 && n != null && ph != null) {
            System.out.println("User: " + n);
            System.out.println("Email: " + e);
            System.out.println("Sending welcome email to " + e);
            System.out.println("Sending welcome SMS to " + ph);
        }
    }
}
```

Questions: (1) What does this code do? (2) What are all the naming problems? (3) Does the function do ONE thing? (4) Are there any repeated patterns? (5) Rewrite it cleanly.

4. Reading Documentation & Searching Effectively

One of the most important skills you will develop is the ability to find answers yourself. The engineers who progress fastest are not the ones who know the most — they are the ones who find answers most efficiently.

4.1 How to Read Official Documentation

Every programming language, framework, and tool has official documentation. It can look intimidating at first, but it follows a predictable structure once you know what to look for.

Section	What to Do With It
Getting Started / Quick Start	Read this first. It gives you the 20% of knowledge that covers 80% of use cases.
Concepts / Guides	Read when you need to understand WHY something works, not just how.
API Reference	Use as a reference — look up specific methods or properties as needed. Do not try to memorise it.
Examples / Tutorials	The fastest way to understand something new. Find an example that resembles your problem.
Changelog	Read when upgrading versions. Understanding what changed helps you avoid breaking things.

4.2 Effective Search Techniques

The ability to search well is a professional skill. Here is how to get better answers faster.

Search Like an Engineer

Include the technology name in every search:

```
BAD: "how to check if list is empty"
GOOD: "Java ArrayList isEmpty() vs size() == 0"
```

Include the exact error message in quotes:

```
"NullPointerException at line 42" Spring Boot
```

Use Stack Overflow intelligently:

- Read the accepted answer AND the top-voted answer — they are sometimes different
- Check the date: a 10-year-old answer may use outdated approaches
- Read the comments: they often contain important corrections
- Understand the answer before copying it — copying code you do not understand creates future bugs

Rule of thumb: If you copy code and cannot explain every line, you have not solved the problem — you have hidden it.

4.3 AI Tools and Their Limits

Tools like GitHub Copilot, ChatGPT, and Claude can be genuinely useful — but they can also be confidently wrong. In this programme, we have a clear policy:

CiT Policy on AI Tools

AI tools are permitted for:

- Explaining a concept in simpler language
- Generating a first draft that you then understand, test, and own
- Suggesting approaches when you are genuinely stuck after your own research

AI tools are NOT permitted for:

- Submitting AI-generated code you cannot explain line by line
- Replacing your own thinking in exercises and assessments
- Skipping the process of understanding how things work

Why this matters: In a job interview or a production incident, there is no AI to bail you out. The goal of this programme is to make YOU capable — not to make you dependent on a tool.

5. Time Management & Professional Communication

Being technically skilled is only half of being a great engineer. How you manage your time and communicate with your team is equally important — especially when things go wrong.

5.1 The 1-Hour Rule

When you are stuck on a problem, you should:

1. Spend up to one focused hour trying to solve it yourself. Use documentation, search, and re-read your code carefully.
2. If you are still stuck after one hour, ask for help — clearly and specifically.

How to Ask for Help Effectively

A bad question:

```
"My code is broken, please help."
```

A good question includes:

- **What you are trying to do:** "I am trying to fetch a list of users from the API and display them in a table."
- **What you expected to happen:** "I expected the table to render with 5 rows."
- **What actually happened:** "The table is empty and the console shows a 404 error."
- **What you have already tried:** "I checked the API URL, it is correct. I logged the response and it is undefined."

This is not just polite — it is more likely to get you a fast, useful answer. And often, writing it out this clearly helps you solve it yourself.

5.2 Proactive Progress Communication

In a professional team, silence is not neutral — it is a red flag. If you are blocked, behind, or confused, your job is to communicate that as early as possible.

Situation	What to Do
You are on track	A brief update at check-in: "I completed the login form, currently working on validation."
You are blocked after 1 hour	Immediately message your instructor: describe the problem, what you tried, and ask specifically.
You will miss a deadline	Tell your instructor BEFORE the deadline — not after. Propose a revised timeline.
You completed something early	Communicate that too — offer to help a peer or ask for a stretch challenge.
You do not understand something	Ask during the session, not at home alone. No question is a bad question here.

5.3 Session Discipline

How you show up to learning matters as much as the content you learn.

CiT Session Standards

- **Arrive on time:** Late arrivals disrupt the session and signal to your peers that their time is less important.
- **Phone on silent:** Notifications steal your deep focus. Give two hours fully.
- **Take notes:** Writing things down helps you retain them. Your future self will thank you.
- **Ask questions out loud:** If you are confused, at least three other people in the room are too. Be the one who asks.
- **Do the exercises:** Watching someone code is not the same as coding yourself. The exercises are not optional.

6. Git Fundamentals

Git is the industry-standard tool for tracking changes in code. Every professional developer uses Git every day. Mastering it early is one of the best investments you can make in your engineering career.

6.1 What is Git and Why Does It Matter?

Think of Git as a time machine for your code. It lets you:

- Save snapshots (commits) of your code at any point in time
- Go back to a previous version if something breaks
- Work on different features simultaneously (branches)
- Collaborate with teammates without overwriting each other's work
- Show your work history to employers (your GitHub profile is your public portfolio)

Beginner's Example: Git as a Video Game Save System

Imagine you are playing a difficult video game. Before every boss fight, you save your progress. If you die, you reload from the save point — you do not start from the beginning.

Git works the same way. Every commit is a save point. If you break your code, you can reload to the last commit. If you want to try something risky, you create a branch — a parallel save — so the original is always safe.

6.2 Core Git Commands

These are the commands you will use every single day. Learn them until they are muscle memory.

Command	What It Does	When You Use It
git init	Creates a new Git repository in the current folder	Once, at the start of a new project
git clone <url>	Downloads a repository from GitHub to your computer	When starting work on an existing project
git status	Shows which files have changed since the last commit	Before every commit — always run this first

git add <file>	Stages a file for the next commit	After making changes you want to save
git add .	Stages ALL changed files	Use with caution — review with git status first
git commit -m "msg"	Creates a save point (commit) with a description	Whenever you complete a small, working change
git push	Uploads your commits to GitHub	After committing, to back up and share your work
git pull	Downloads the latest changes from GitHub	Before starting work each session
git log --oneline	Shows a compact history of all commits	To review your save history
git diff	Shows exactly what changed in unstaged files	Before staging — to review your changes

6.3 Writing Good Commit Messages

A commit message is a note to your future self and your teammates. A bad message wastes everyone's time. A good message is a gift.

Commit Message Examples

BAD messages:

```
git commit -m "fix"
git commit -m "changes"
git commit -m "asdfgh"
git commit -m "final version 3"
```

GOOD messages:

```
git commit -m "add user registration endpoint with email validation"
git commit -m "fix NullPointerException in TransactionService.transfer()"
git commit -m "refactor: extract discount calculation into DiscountService"
```

Format: Use the imperative tense: "add feature", not "added feature" or "adding feature". Describe WHAT changed and WHY if it is not obvious.

6.4 Branching & Pull Requests

A branch is an independent line of development. You create a branch when working on a new feature, so the main codebase stays clean and working.

The Professional Git Workflow

Step 1: Start from the main branch and get the latest code.

```
git checkout main
git pull
```

Step 2: Create a new branch for your feature.

```
git checkout -b feature/user-registration
```

Step 3: Make your changes and commit regularly.

```
git add .
git commit -m "add POST /api/users endpoint with validation"
```

Step 4: Push your branch to GitHub.

```
git push -u origin feature/user-registration
```

Step 5: Open a Pull Request on GitHub. Describe what you changed and why. Request a reviewer.

Rule: Never push directly to main. Always use a branch and a pull request — even when working alone. This is the industry standard.

Exercise 3: Your First Git Workflow (Hands-On)

Complete this exercise in your terminal. Take a screenshot of your terminal after each major step.

Part A — Create a Repository:

3. Create a new folder called cit-week2-practice and navigate into it
4. Run git init to initialise a Git repository
5. Create a file called HelloCiT.java and add a comment or a line of text inside it
6. Run git status — what do you see?
7. Run git add . then run git status again — what changed?
8. Run git commit -m "initial commit: add HelloCiT.java"

Part B — Branching:

9. Create a new branch: `git checkout -b feature/my-first-feature`
10. Make a change to your file (add another line or comment)
11. Commit the change with a descriptive message
12. Switch back to main: `git checkout main` — notice your change is gone (it's on the branch!)
13. Switch back to your feature branch: `git checkout feature/my-first-feature` — your change is back

Part C — GitHub:

14. Create a new repository on GitHub.com (click the + icon)
15. Follow the instructions to link your local repo: `git remote add origin <your-github-url>`
16. Push your main branch: `git push -u origin main`
17. Push your feature branch too
18. On GitHub, open a Pull Request from your feature branch into main. Write a description of your change.

Share your GitHub repository link with your instructor when done.

7. Key Takeaways & Next Steps

7.1 What You Learned This Week

- Problem decomposition: breaking large problems into small, solvable steps
- Pseudocode: planning logic in plain English before writing any code
- DRY principle: every piece of logic lives in exactly one place
- Meaningful naming: code that reads like prose needs fewer comments
- Clean code habits: short functions, no magic numbers, formatted consistently
- SOLID principles: a framework for designing code that is easy to maintain and extend
- Reading documentation: how to navigate official docs and search effectively
- The 1-Hour Rule and how to ask for help clearly and professionally
- Git core workflow: init, add, commit, push, pull, branch, and pull request

7.2 Homework / Self-Study

Complete these tasks before the next session:

19. Complete Exercise 3 (Git hands-on) if you did not finish it in class — share your GitHub link
20. Create a GitHub profile if you have not already. Pin your `cit-week2-practice` repository.
21. Revisit the pseudocode exercise from Section 2 and write clean pseudocode for all three features
22. Read: "How to Write a Git Commit Message" by Chris Beams (search for it — it is a classic article)
23. Preview Week 3: Install IntelliJ IDEA Community Edition if you have not done so, and familiarise yourself with creating a new Java project

7.3 Looking Ahead — Week 3

Next week, **Jim & Anderson** will take you through **Java Fundamentals**. Come with your environment set up and your Git skills ready — you will be committing real code from day one.

Closing Reflection

As we close this session, let us carry with us the reminder that excellence is not an accident — it is a habit. Proverbs 22:29 tells us:

"Do you see someone skilled in their work? They will serve before kings; they will not serve before officials of low rank."

— Proverbs 22:29 (NIV)

Commit to excellence in every line of code, every commit message, and every conversation with your teammates. Go forth and build with purpose!

Glossary of Terms

Term	Definition
Algorithm	A defined sequence of steps that solves a specific problem
Branch	An independent line of development in Git — a parallel copy of the code to work in safely
Clean Code	Code that is readable, maintainable, and understandable without explanation
Commit	A saved snapshot of your code changes in Git — like a save point
DRY	Don't Repeat Yourself — a principle stating that every piece of logic should exist in exactly one place
Git	A version control system that tracks changes to code over time
GitHub	An online platform for hosting Git repositories and collaborating with other developers
Problem Decomposition	The skill of breaking a large, complex problem into smaller, manageable steps
Pseudocode	A plain-English description of code logic, written before actual code is created
Pull Request (PR)	A request to merge changes from one branch into another — used for code review
Refactoring	Improving the structure and readability of existing code without changing what it does
Repository (Repo)	A folder tracked by Git that contains your project and its full history
SOLID	Five design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
Version Control	A system for tracking and managing changes to files over time